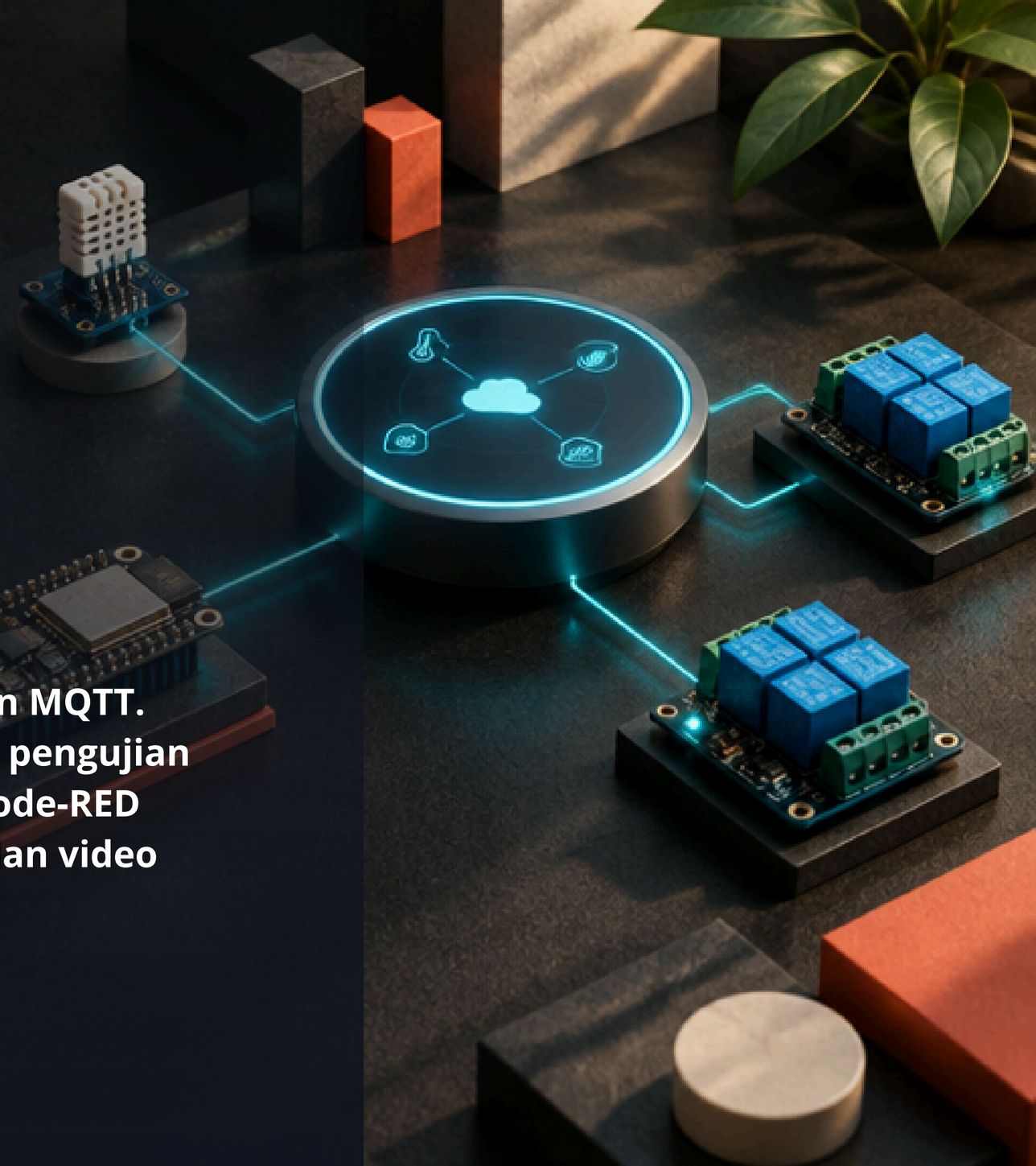
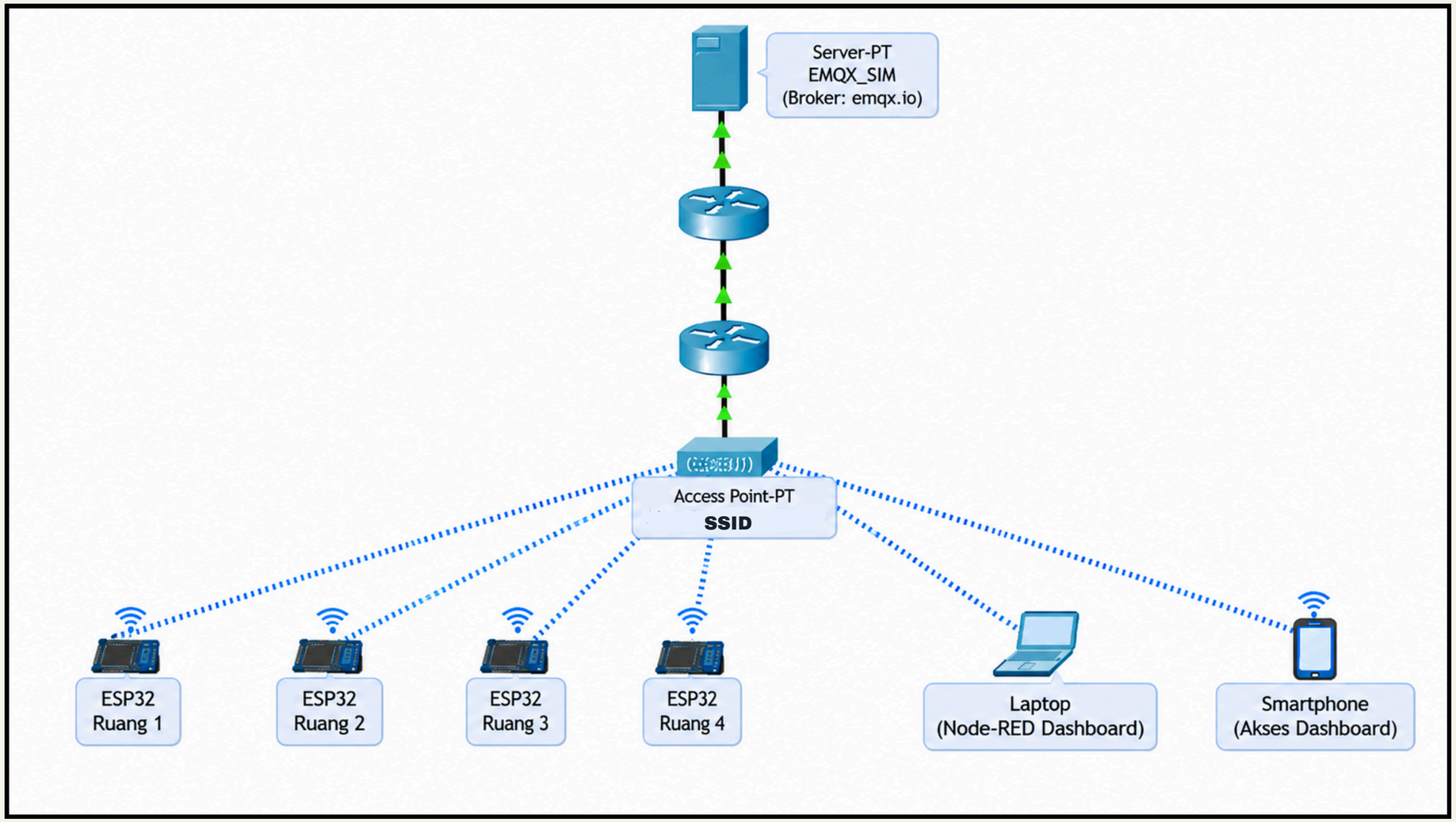
Dashboard Node-RED



Kelompok :

1. **M. Akmalul Ihsan (2124600093) - integrasi ESP32 dan MQTT.**
2. **M. Jefri Al Bukhori (2124600098) - dokumentasi dan pengujian**
3. **Ahmad Syahrial Karim (2124600100) - dashboard Node-RED**
4. **Lailiah Ratu Nova (2124600104) - integrasi GitHub dan video**

Arsitektur Sistem dan Alur Data



Rancangan Topic MQTT Final Sesuai Kode

Topic MQTT	Arah	Payload / Keterangan	QoS	Retain
pens/klp12/ruangX/temperature	ESP32 → Node-RED	Data suhu dari sensor DHT11, format 1 angka di belakang koma (contoh: 28.5 °C)	0	TRUE
pens/klp12/ruangXhumidity	ESP32 → Node-RED	Data kelembapan dari sensor DHT11, format 1 angka di belakang koma (contoh: 65.0 %)	0	TRUE
pens/klp12/ruangX/lamp1/set	Node-RED → ESP32	Perintah lampu 1: 1 = ON/nyala, 0 = OFF/mati	1	-
pens/klp12/ruangX/lamp1/state	ESP32 → Node-RED	Status aktual lampu 1: 1 = ON/nyala, 0 = OFF/mati	0	TRUE
pens/klp12/ruangX/lamp2/set	Node-RED → ESP32	Perintah lampu 2: 1 = ON/nyala, 0 = OFF/mati	1	-
pens/klp12/ruangX/lamp2/state	ESP32 → Node-RED	Status aktual lampu 2: 1 = ON/nyala, 0 = OFF/mati	0	TRUE
pens/klp12/ruangX/pstatus	ESP32 → Broker / Node-RED (status LWT dari broker saat koneksi terputus tidak normal)	online = ESP32 terhubung; offline = pesan LWT ketika koneksi terputus tidak normal	1	TRUE

Catatan : ruangX (ruang1, ruang 2, ruang 3, ruang 4)

Last Will and Testament (LWT)

```
bool berhasil = mqtt.connect(
    clientId.c_str(),   // Client ID unik (dari MAC)
    tStatus,            // Topik will: pens/klp12/ruang2/pstatus
    1,                  // QoS will = 1
    true,               // Retain will = true
    "offline"           // Pesan will
);

// Setelah connect berhasil:
mqtt.publish(tStatus, "online", true);
```

LWT adalah pesan yang dititipkan ESP32 ke broker saat connect. Jika ESP32 putus tiba-tiba (listrik mati, Wi-Fi hilang), broker otomatis mengirim offline ke topik pstatus.

Saat berhasil connect, ESP32 mengirim online ke topik yang sama. Retain true membuat dashboard yang baru dibuka langsung tahu status terakhir perangkat.

Logika reconnect sistem

```
if (WiFi.status() != WL_CONNECTED) {
    unsigned long now = millis();
    if (now - lastWiFiAttempt >= WIFI_INTERVAL) {
        lastWiFiAttempt = now;
        Serial.println("Wi-Fi terputus. Mencoba reconnect...");
        WiFi.reconnect();
    }
    return;
}
```

```
if (!mqtt.connected()) {
    reconnectMQTT();
    return;
}
mqtt.loop();

void reconnectMQTT() {
    unsigned long now = millis();
    if (now - lastConnectAttempt < CONNECT_INTERVAL) return;
    lastConnectAttempt = now;

    // connect() dengan LWT ...

    if (berhasil) {
        mqtt.subscribe(tL1Set, 1);
        mqtt.subscribe(tL2Set, 1);
        mqtt.publish(tStatus, "online", true);
        publishLampState();   // sinkronisasi status lampu
        lastPublish = millis();
    }
}
```

Keterangan :

- **Reconnect berjalan non-blocking memakai millis(), bukan delay(), sehingga program tidak macet.**
- **Urutan pengecekan: Wi-Fi dulu, baru MQTT.**
- **Percobaan ulang dilakukan tiap 5 detik.**
- **Setelah tersambung kembali, ESP32 melakukan subscribe ulang, mengirim status online, dan menyinkronkan status lampu.**

Callback Kendali Lampu

```
void callback(char* topic, byte* data, unsigned int len) {
    String payload;
    payload.reserve(len);

    for (unsigned int i = 0; i < len; i++) {
        if (data[i] != '\n' && data[i] != '\r' &&
            data[i] != ' '  && data[i] != '\t') {
            payload += (char)data[i];
        }
    }

    if (payload != "1" && payload != "0") {
        Serial.println("Perintah ditolak. Gunakan payload 1 atau 0.");
        return;
    }

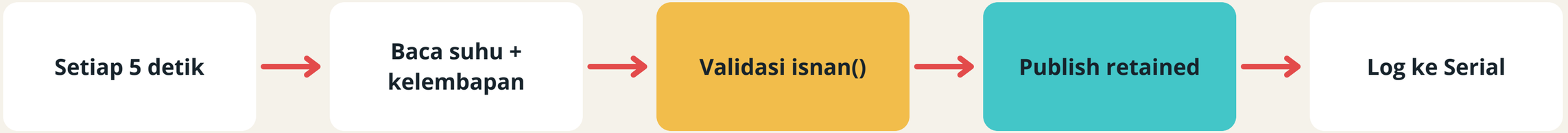
    bool nyala = (payload == "1");

    if (strcmp(topic, tL1Set) == 0) {
        digitalWrite(L1_PIN, nyala ? HIGH : LOW);
        publishLampState();
    }
    else if (strcmp(topic, tL2Set) == 0) {
        digitalWrite(L2_PIN, nyala ? HIGH : LOW);
        publishLampState();
    }
    else {
        Serial.println("Topic tidak dikenali.");
    }
}
void publishLampState() {
    if (!mqtt.connected()) return;
    const char* state1 = (digitalRead(L1_PIN) == HIGH) ? "1" : "0";
    const char* state2 = (digitalRead(L2_PIN) == HIGH) ? "1" : "0";
    mqtt.publish(tL1State, state1, true);
    mqtt.publish(tL2State, state2, true);
}
```

Keterangan :

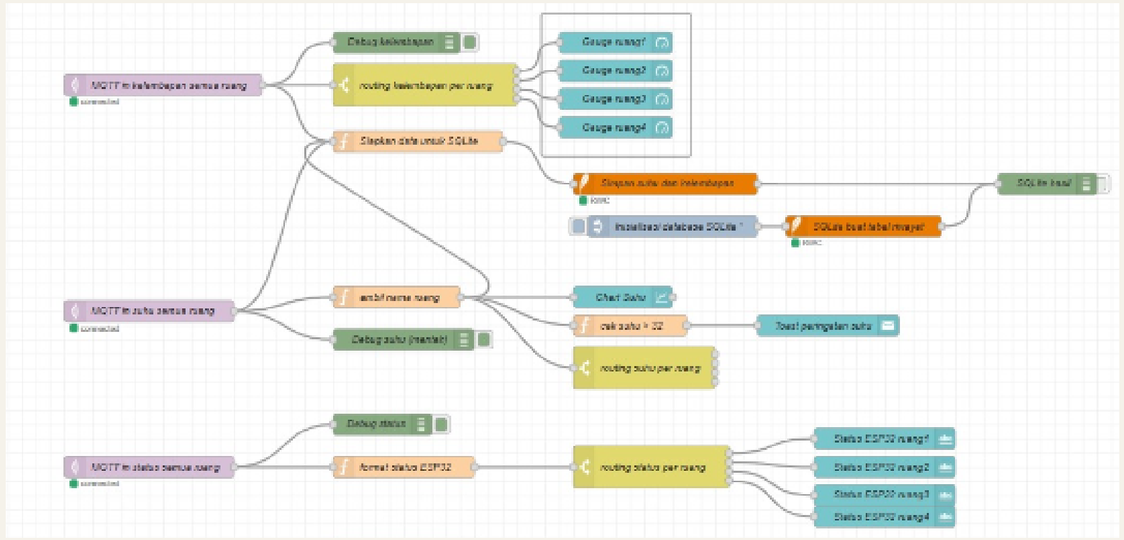
- **Callback dipanggil otomatis setiap ada pesan masuk di topik lamp1/set atau lamp2/set.**
- **Payload dibersihkan dari spasi dan newline, lalu divalidasi: hanya 1 (ON) dan 0 (OFF) yang diterima.**
- **Topik dicocokkan dengan strcmp untuk menentukan lampu mana yang dikontrol.**
- **ESP32 mengirim status terbaru ke lamp1/state dan lamp2/state sebagai feedback ke dashboard.**

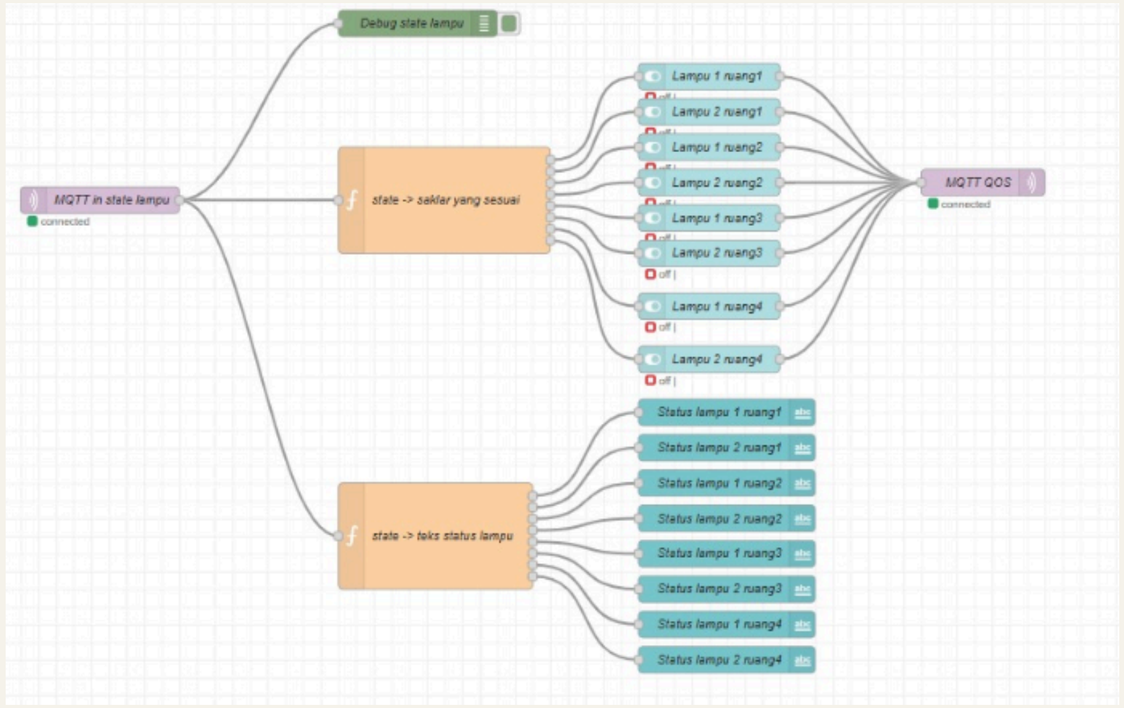
Pembacaan DHT11 & Publikasi Data



`temperature` • `humidity`

Flow Node-RED: Bagian Atas + Bawah

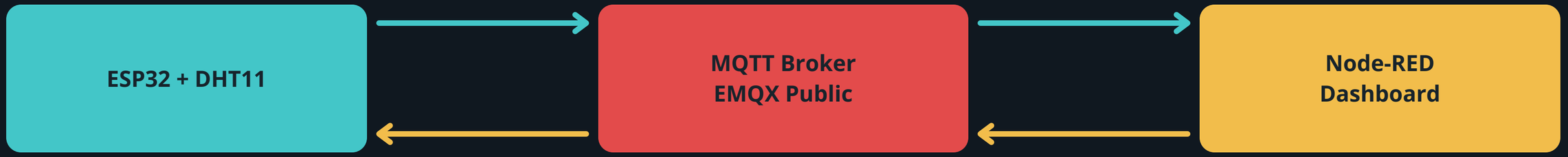




- **Bagian kiri (Monitoring): MQTT In menerima data sensor dari ESP32, kemudian Function mengolah data untuk ditampilkan melalui Gauge dan Chart. Debug digunakan untuk memeriksa data yang masuk.**

- **Bagian kanan (Kontrol): MQTT In menerima data status lampu, sedangkan Switch digunakan untuk mengontrol ON/OFF delapan lampu dari empat ruangan. Function mengolah perintah dan status, kemudian MQTT Out mengirimkan perintah ke ESP32.**

Skenario Komunikasi Data & Kendali



LWT: broker menerbitkan offline bila ESP32 terputus mendadak

Kendala dan Solusi Sistem

Dua ESP32 saling putus-sambung	clientId unik berbasis MAC • topik per ruang • cek stabilitas Wi-Fi • reconnect non-blocking
Chart hanya satu garis	Pisahkan topik temperature dan humidity, atau konfigurasi multi-series pada chart Node-RED
Status offline lama muncul	Retained LWT yang benar • publish online retained • pstatus per ruang • tangani retained subscriber

Catatan produksi: broker publik cocok untuk demo; sistem produksi memerlukan autentikasi dan kontrol akses.

Sekian, Terima kasih